Test report

MIRAI TRACKER · 20 OCTOBER 2026

Mirai Course Progression Tracker. Evidence date: 20 October 2026. Code baseline: 43ea893 plus the submission packaging and demo-label changes recorded in Git.

Scope and outcome

The test scope covers administrator, program manager, campus manager, faculty and student access; calendar/timetable propagation; lesson plans and logs; calculations; exams; reports; notifications; imports; and scale. The requested deployment has three campuses: HI-Tech, Jaggannath and Rathinam.

Evidence	Result
Unit/component tests	586 passed: API 408, web 147, calculations 23, worker 6, validation 2
PostgreSQL integrations	27 passed across six suites
Authenticated HTTP permissions	11 passed using password and Mailpit OTP
Independent calculations	18 of 18 numeric comparisons match to two decimals
Concurrent load	50, 100 and 200 users, zero HTTP/payload errors
Live demonstration	3 campuses, 15 faculty, 15 subjects, 540 lecture logs

A regression check also verifies that a larger semester download can complete beyond the normal 15-second read deadline; downloads have a bounded 60-second budget.

The two stale integration fixtures were updated for Mirai/University terminology and the new buffer-threshold context. Their affected suite then passed against a fresh migrated database.

Roles and permissions

Role	Check	Outcome
Admin	Administrator sees all 30 subjects	HTTP 200; passed
Program Manager	Program manager sees all 30 subjects	HTTP 200; passed
Campus Manager	Campus manager sees only own 10 subjects	HTTP 200; passed
Campus Manager	Campus manager cannot read another campus subject	HTTP 403; passed
Faculty	Faculty sees only assigned subject	HTTP 200; passed
Faculty	Faculty cannot read another faculty subject	HTTP 403; passed
Faculty	Faculty cannot log another faculty class	HTTP 403; passed
Student	Student can read own overview	HTTP 200; passed
Student	Student cannot log teaching	HTTP 403; passed
Student	Student cannot create progress snapshot	HTTP 403; passed
Admin	Administrator cannot use faculty logging route	HTTP 403; passed

Additional database integrations exercise scoped reports/exports, campus calendar edits, bulk access, manual semester-to-subject linkage and AI conversation ownership. A student's controller is read-only. Faculty are blocked from another faculty member's subject and log route, including direct API attempts.

Calculation accuracy

Expected values are independently summed from topic and future-session database rows, then compared with the authenticated dashboard API. The expected calculation does not call the shared analytics helper. The accompanying Excel workbook exposes the formulas and the saved API values.

Subject	Coverage %	Remaining h	Available h	Buffer	Result
Data Structures and Algorithms	81.67	8.25	20.00	2.42	Match
Web Development	59.58	24.25	20.00	0.82	Match
Database Systems	46.33	40.25	20.00	0.50	Match

Scope, covered hours, coverage percentage, remaining hours, available hours and buffer ratio are all checked. The earlier hand-worked CSV separately exercises ON TRACK, TIGHT and DANGER. Threshold boundaries use raw ratios before display rounding.

Calendar and exam evidence

Calendar-engine tests select exactly the second and fourth Saturdays across three months, reject fifth Saturdays, and apply explicit holiday/working-day overrides. Schedule-engine tests show every affected later topic shifting after cancellation or a missing holiday slot. Service tests check regeneration after holiday creation, editing, deletion and semester-rule changes.

Recovery tests reject closed dates and conflicts, preserve logged history and prevent already-resolved cancellations from moving again. Exam tests calculate only mapped topics, compare mid-term/end-term scopes and enforce deadlines. The live demo has two scoped exam windows per campus.

Logging and alerts

Fictional account	Measured automated flow (seconds)	Result
Fictional faculty 1	0.07	Passed
Fictional faculty 2	0.04	Passed
Fictional faculty 3	0.04	Passed
Fictional faculty 4	0.04	Passed
Fictional faculty 5	0.04	Passed

Median automated list-and-save time: **0.04 seconds**. These are five fictional-account HTTP workflows with actual timings. They measure API responsiveness, not human reading, typing or phone usability; no human-volunteer timing is claimed.

Alert tests verify end-of-campus-day faculty reminders, the exact 48-hour escalation boundary, deduplication, retries and cancellation of pending messages after logging. SMTP behavior is tested with local capture or mocks. External delivery is not inferred from those tests.

Imports and connected workflows

The current interface has separate calendar, batch timetable, lesson-plan and student-access imports. Parser and service tests cover row preservation, invalid dates, partial plans, worksheet selection, errors and relationships. The retired full-semester backend importer has an integration test; that is not a claim that the removed interface is available.

The full scale seed is loaded through a dedicated additive evaluator script, not a CSV import. Thus the large-dataset exercise proves storage and application behavior with 30 subjects, not a 30-subject CSV import acceptance result.

Load and live data

Clients	Requests	Dashboard p95 ms	Logging p95 ms	Error rate
50	300	289.92	541.93	0.00%
60	360	335.02	781.14	0.00%
100	600	570.31	1062.91	0.00%
200	1200	1046.38	1998.97	0.00%

The separate burst used 60 distinct authenticated faculty, one submission each: p95 **264.86 ms**, 0 errors. All completed in 0.27 seconds. This is a simultaneous burst within the requested 15-minute window, not a sustained 15-minute soak.

The live additive seed preserves existing records and provides 450 topics, 885 classes, 540 logs, 6 exams and 39 dated overrides. Demo notifications and calculated weekly previews make their respective interfaces inspectable. Visiting-faculty rates, expenses and payments are fictional examples.

Reproduction and evidence boundaries

Run `pnpm test`, the six files in `apps/api/test` using `jest.database.config.cjs` against disposable migrated databases, and the scripts `submission-load-test.mjs` and `submission-calculations.mjs` with private local manifests. The seed script has a local scale profile and a separately guarded live demo profile.

Evidence is stored in `docs/evidence/2026-10-20`. Load measurements include response-body validation and real HTTP authentication, PostgreSQL and Redis. Browser layout is checked separately; API timing does not measure page paint. Production capacity, a sustained soak, external WhatsApp delivery and human phone-task duration are outside these measured results.

Scalability and load report

MIRAI TRACKER · 20 OCTOBER 2026

Measured on 2026-10-20T14:37:26.735Z. Three campuses are used as requested.

Dataset and environment

Item	Configuration
Academic dataset	3 campuses; 60 faculty; 30 subjects; 1,020 students; full semester relations
Host	Apple M5; 10 logical CPUs; 16 GiB RAM
Runtime	v20.18.0; darwin arm64
Persistence	Real PostgreSQL 16 and Redis 7; Prisma connection pool 10
Authentication	68 accounts authenticated through password and Mailpit email OTP
Network	Loopback HTTP; production Vercel/Neon latency is not represented

The supported setup runtime is Node 22.13+. This measurement run used the available Node 20.18.0 host runtime; reproduce on the supported runtime before comparing deployment capacity.

Workload

Each virtual user performs three sequential dashboard-read/log-write pairs. Virtual users run concurrently. Ten percent of dashboard readers use the administrator's full 30-subject scope; the remainder use assigned faculty scope. Writes use distinct class identifiers within each level and create real log revisions. Authentication is completed before timed load.

This is a short burst benchmark. It measures client-observed HTTP completion including JSON parsing. Success requires both the expected status and payload shape. It does not use the earlier mocked demo API benchmark.

Clients	Requests	Dashboard p95 ms	Logging p95 ms	Error rate
50	300	289.92	541.93	0.00%
60	360	335.02	781.14	0.00%
100	600	570.31	1062.91	0.00%
200	1200	1046.38	1998.97	0.00%

Target checks

The 60-distinct-faculty burst recorded p95 logging latency of **264.86 ms**, below the 1,000 ms target, with zero errors. All 60 submissions completed in 0.27 seconds, inside the 15-minute window.

Five sequential administrator dashboard requests across all 30 subjects recorded p95 **86.71 ms**, below the 2,000 ms API target. Mixed-load dashboard p95 remained below 2,000 ms at every level. Browser rendering and internet transit are not included.

At 100 and 200 concurrent mixed-load clients, logging p95 exceeds 1,000 ms. The results establish the observed local limit rather than claiming every stress level meets that latency target. All requests still completed without HTTP or payload errors.

Scaling plan

Keep PostgreSQL and Redis external to stateless API instances. Match connection pools to the database's total connection budget; adding replicas without budgeting connections can worsen queueing. Move long AI/file work and scheduled email processing away from interactive requests.

Profile the class-context queries and transaction wait time first under heavy logging. Cache or preaggregate dashboard results only with campus/subject permission keys and invalidation on logs, plans, calendars and assignments. Retain strong transaction boundaries and conflict checks.

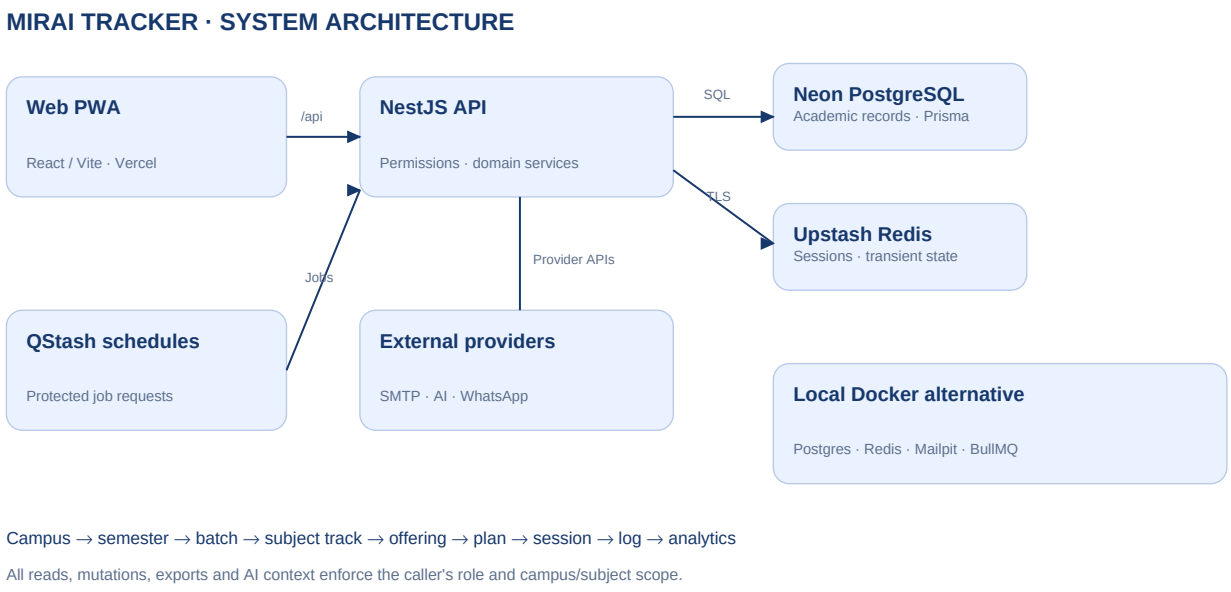
For a hosted capacity claim, rerun the same authenticated workload on an isolated staging deployment with production-equivalent Vercel limits, Neon region/tier and Upstash configuration. Add a sustained 15-minute run, cold-start samples and browser page-load measurements. Track p50/p95/p99, error rates, database connections and slow queries.

Reproduction

Create a fresh migrated local database, run `apps/api/prisma/seed-evaluator.ts` with `EVALUATOR_PROFILE=scale`, start the API, then run `node scripts/submission-load-test.mjs <private-env.json> <private-accounts.json>`. The script rejects a remote database target. Results and CSV are committed without credentials.

Architecture and design

MIRAI TRACKER · 20 OCTOBER 2026



Purpose and scope

Mirai Tracker connects planning to recorded teaching for HI-Tech, Jaggannath and Rathinam. A semester belongs to a campus. A batch contains offerings in Mirai and University tracks. Shared records drive dashboards, reports, faculty plans and the student view.

Deployment

React/Vite serves the PWA from Vercel. Same-origin /api requests reach NestJS on Vercel. Prisma accesses Neon PostgreSQL. Upstash Redis stores opaque sessions and transient auth/job state. QStash invokes protected scheduled endpoints. SMTP delivers OTPs and summaries; AI and WhatsApp are separately configured providers.

Local Compose runs the web, API, PostgreSQL, Redis, Mailpit and BullMQ worker. The load report uses a host API with local containerized data services. It does not claim to reproduce hosted network latency.

Data and authorization

Users have campus-scoped roles. Faculty profiles link to offerings through dated assignments. Students link to a batch. Campus → semester → batch → offering → lesson plan → unit → topic forms the academic hierarchy. Timetable entries generate sessions; revisioned logs record topic updates. Exam windows map to exact topics.

Administrators/program managers oversee all campuses. Campus managers manage their campuses. Faculty work with assigned subjects. Students read their enrolled batch. Server-side guards and row scoping apply to reads, writes, reports, exports and AI context.

Calculation and calendar

Topic minutes are canonical. Covered minutes include completed topics and weighted partial completion. Remaining minutes equal scope minus covered minutes. Available minutes come from eligible future sessions through the deadline. Buffer is available divided by remaining. Default thresholds are 1.15 for ON TRACK and 1.00 for TIGHT; complete and missing-plan states are distinct.

Explicit calendar entries override weekly rules. Sundays and non-working Saturdays are excluded unless opened. Timetable generation checks conflicts and preserves logged history. Changes produce revised planned dates; affected classes can require faculty review before a recovery move.

Imports, delivery and reports

CSV/XLSX rows are parsed, validated and previewed before domain writes. AI helps with unstructured calendars and ambiguous mappings. Separate pages import calendars, batch timetables, lesson plans and student access. The full-semester import interface is retired.

Logs record completion, classwork/homework and topic changes. Reports use shared scoped analytics. Exports retain structured records. Visiting-faculty honoraria derive from recorded classes and effective per-class rates, with expenses/payments shown separately.

Security and operations

Passwords use salted scrypt. OTPs expire after ten minutes with attempt/request limits. Session cookies are HTTP-only, SameSite and Secure in production. Authorization is refreshed from PostgreSQL. Validation, origin checks and safe errors protect mutations. AI receives scoped data and output is rendered without raw HTML execution.

Transactions protect related writes. Unique job/revision keys and leases reduce duplicate processing. Scheduler secrets protect internal routes. Audit records retain actor and scope. Secrets and demo passwords stay outside Git and Docker build contexts.

The scale fixture contains three campuses, 60 faculty, 30 subjects and 1,020 students. The separate report records 50/100/200-client loads, a 60-faculty burst and full-semester reads. Pool budgeting, permission-safe aggregation and hosted sustained tests guide further scaling.